

R Lab Week 1 Part 1

Robert Gulotty

9/30/2016

Intro to data.frame

In this document, we will learn how to load and manipulate data frames, the objects that store datasets in R. The way to read this document is that the commands in the boxes are R code. The things to the right of the `##` below indicates the output of these commands. In Rstudio, the suggested platform for using R, this code is typed out in a **script**, and then run in the console by selecting the code and typing **command**'enter'. For more general help, type `help.start()` in the console and click enter. For the r-reference card/ cheat sheet, go to <https://cran.r-project.org/doc/contrib/Short-refcard.pdf>

Preliminaries, loading packages

The first time you run R, you will have to install packages. These are “libraries” that contain interesting code and data for us to use. You install them by using a function called `install.packages`. The first thing in the `install.packages` function is the name of the package. Below you will see code that installs ‘dplyr’, ‘ggplot2’ and ‘nycflights’. Notice, the packages are case sensitive and surrounded by quotation marks. The next thing listed within the parenthesis is generally omitted, but if specified, tells the computer to look for these packages in the rstudio repository. CRAN, the R organization, has many repository options, and if you leave off this part, R will use your default repository, set the first time you run a install command. The third part, setting dependencies to `TRUE`, asks R to redownload all of the assumed packages that the installed package requires. Again, I usually leave this out, but there are cases where a package is not updating properly, and this is a quick fix.

```
install.packages("dplyr", repos="http://cran.rstudio.com/", dependencies = TRUE)
install.packages("ggplot2", repos="http://cran.rstudio.com/", dependencies = TRUE)
install.packages("nycflights13", repos="http://cran.rstudio.com/", dependencies = TRUE)
```

You will should only have to run each of these commands once and when you restart R it will still be downloaded and installed. It is never a problem to re-run a command in R.

After installation, you can load the packages with the `library` command. This function just uses the name of the package, with or without quotation marks. Notice the warning after you load the dplyr package. When a package is loaded, it introduces a number of functions that may have the same name as functions already loaded in R. In this case, `filter` and `lag` were functions in the preloaded `stats` package, but we are rewriting it with the dplyr `filter` and `lag` command. If you want to use a function with a redundant name, you can always use `::` to make sure it is coming from the right package. For example, `dplyr::filter()` or `stats::filter()`.

```
library(ggplot2)
library(nycflights13)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
```

```
##
##      filter, lag
## The following objects are masked from 'package:base':
##
##      intersect, setdiff, setequal, union
```

If you want to see all the functions that come with a package, check out the help file by putting a specific command in the regular help function `help(package="ggplot2")`. Online you can find many resources at the official CRAN website, which organizes packages by topic.

Data Description functions `dim()`, `head()`, `str()`

R is a language of functions. Just as in mathematics, functions can transform some input in a way that we find interesting. A function might summarise some data, or it might produce a plot.

On the screen, functions look like words with a `()` next to them. The things within the `()` are the inputs to the function, or in R-speak, the **Arguments** of the function. The output of the function is called the **Value** of the function. We have already used two special examples of functions, `library()` and `install.packages()`.

To check your understanding, what would the argument be for a function that finds the distance between two cities? What might a value of such a function be?

Today we are going to work with functions on an amazing dataset, `flights`, which describes all the flights out of New York in 2013.

We begin by identifying the **dimensions** of the `flights` dataset, which is already loaded into R with the `nycflights` library using the `dim` function.

```
dim(flights)
```

```
## [1] 336776      19
```

What do you think these numbers refer to? A little heuristic, whenever dealing with a matrix or dataframe is that the order is Rows and Columns, or Rum and Coke, Roman Catholic. $R \times C$. In this case, we have 336,776 rows, and 19 columns. The rows refer to a flight, the columns refer to a characteristic or measured attribute of that flight.

When using data from libraries, you can use the `?` command, as follows : `?flights`

The most common function that I use is one that glimpses the top of a dataset, called `head()`

```
head(flights)
```

```
## # A tibble: 6 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>
## 1  2013     1     1     517             515           2     830
## 2  2013     1     1     533             529           4     850
## 3  2013     1     1     542             540           2     923
## 4  2013     1     1     544             545          -1    1004
## 5  2013     1     1     554             600          -6     812
## 6  2013     1     1     554             558          -4     740
## # ... with 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dtm>
```

```
# head(flights, n=8)
```

Note that you get the first 6 rows by default. You can change this by adding to the head command an additional parameter “n”.

A second useful examination function is `str()`, which gives the structure of the object.

```
str(flights)

## Classes 'tbl_df', 'tbl' and 'data.frame':   336776 obs. of  19 variables:
## $ year      : int  2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
## $ month     : int   1  1  1  1  1  1  1  1  1  1 ...
## $ day       : int   1  1  1  1  1  1  1  1  1  1 ...
## $ dep_time  : int  517 533 542 544 554 554 555 557 557 558 ...
## $ sched_dep_time: int  515 529 540 545 600 558 600 600 600 600 ...
## $ dep_delay : num   2  4  2 -1 -6 -4 -5 -3 -3 -2 ...
## $ arr_time   : int  830 850 923 1004 812 740 913 709 838 753 ...
## $ sched_arr_time: int  819 830 850 1022 837 728 854 723 846 745 ...
## $ arr_delay  : num  11 20 33 -18 -25 12 19 -14 -8 8 ...
## $ carrier    : chr  "UA" "UA" "AA" "B6" ...
## $ flight     : int 1545 1714 1141 725 461 1696 507 5708 79 301 ...
## $ tailnum    : chr  "N14228" "N24211" "N619AA" "N804JB" ...
## $ origin     : chr  "EWR" "LGA" "JFK" "JFK" ...
## $ dest       : chr  "IAH" "IAH" "MIA" "BQN" ...
## $ air_time   : num  227 227 160 183 116 150 158 53 140 138 ...
## $ distance   : num 1400 1416 1089 1576 762 ...
## $ hour       : num   5  5  5  5  6  5  6  6  6  6 ...
## $ minute     : num  15 29 40 45  0 58  0  0  0  0 ...
## $ time_hour  : POSIXct, format: "2013-01-01 05:00:00" "2013-01-01 05:00:00" ...
```

Notice the first line of output gives the “class” of `flights`. Here `flights` has three types, `tbl_df`, `tbl` and `data.frame`. It then lists the structure of the content of the data. Notice, R has several basic data types:

- **int** refers to integers, such as 22, -9 and 0.
- **num** refers to real numbers, such as .03, 19, and π .
- **chr** refers to characters strings, or letter based data, such as “Alabama”, “horseradish”, and “C-3PO”.
- **POSIXct** is a date format, which we will learn about later.
- **logi** (not listed here) refers to the boolean logical statement, TRUE and FALSE, manipulated by the logical commands described in the next section.
- **Factor** (not listed here) refers to a categorical variable, a special encoded character vector.

You can switch between these data types using `as.numeric()`, `as.character()`, `as.integer()`, `as.logical()`, and `as.factor()`. Sometimes R will allow you to switch between data types without explicitly saying that you want to. This is very useful for applied work.

Dplyr Commands and Logic

The first command to learn in the dplyr framework is `filter`. `filter` takes on three arguments. The first is a data frame, which has all your data in it. After that, you can put any number of conditions that you want to query from that dataset. The function grabs the subset of rows that satisfy logical conditions listed after the data frame. Here, we are grabbing the flights where on the day is equal to 1 and the month is equal to 1, which is to say all the flights on new years day of 2013.

```
filter(flights, month == 1, day == 1)
```

```
## # A tibble: 842 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>
## 1  2013     1     1     517           515           2     830
## 2  2013     1     1     533           529           4     850
## 3  2013     1     1     542           540           2     923
## 4  2013     1     1     544           545          -1    1004
## 5  2013     1     1     554           600          -6     812
## 6  2013     1     1     554           558          -4     740
## 7  2013     1     1     555           600          -5     913
## 8  2013     1     1     557           600          -3     709
## 9  2013     1     1     557           600          -3     838
##10  2013     1     1     558           600          -2     753
## # ... with 832 more rows, and 12 more variables: sched_arr_time <int>,
## #   arr_delay <dbl>, carrier <chr>, flight <int>, tailnum <chr>,
## #   origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
## #   minute <dbl>, time_hour <dtm>
```

Logical commands

Inside that `filter` command is actually another function, the `==` function. `filter`'s arguments are logical vectors, like `TRUE`, `TRUE`, `FALSE`. The data in `flights` are not logical. As we saw above with the `str` command, the columns of `flights` are all integers, numbers, and characters. There is even a `POSIXct` format variable for time, but no logicals.

In order to use the `filter` function, we need a function which takes as an argument an input something from the `flights` dataset and gives as a value a logical vector. There are a number of these logical functions in R. These, and all standard functions, can be found in help files by typing `?'COMMAND'`.

- `==` refers to 'equal to'. It returns `TRUE` if both sides are equal and not missing.
- `>=` and `<=` refer to 'greater than or equal to' and 'less than or equal to'. These compare numbers.
- `!=` means not equal to.
- `&` is a logical **and**
- `|` is the logical **or**
- `!` is the logical negation.

Here we get all the flights from New York to Chicago.

```
filter(flights, month == 1, day == 1, dest=="ORD" | dest=="MDW")
```

```
## # A tibble: 56 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>
## 1  2013     1     1     554           558          -4     740
## 2  2013     1     1     558           600          -2     753
## 3  2013     1     1     608           600           8     807
```

```
## 4 2013      1      1      629      630      -1      824
## 5 2013      1      1      656      700      -4      854
## 6 2013      1      1      709      700       9      852
## 7 2013      1      1      715      713       2      911
## 8 2013      1      1      739      745      -6      918
## 9 2013      1      1      749      710      39      939
## 10 2013     1      1      828      830      -2     1027
## # ... with 46 more rows, and 12 more variables: sched_arr_time <int>,
## #   arr_delay <dbl>, carrier <chr>, flight <int>, tailnum <chr>,
## #   origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
## #   minute <dbl>, time_hour <dtm>
```

Practice Q. How many funtions are in the above code?

Practice Q. How many flights traveled to Des Moines International from New York in August?

Practice Q. On what day did a flight go to Lexington, KY (LEX)?

At this point you should be able to answer the first four questions in the assignment section of this document.

Base R comparison

In “base” or unmodified R, all this would be more cumbersome, with the following commands used to grab new year’s day flights.

```
januaryflights <- flights[, "month"] == 1
firstofmonthflight <- flights[, "day"] == 1
NewYearslogical <- januaryflights & firstofmonthflight
flights[NewYearslogical,]
```

Although we will use the dplyr commands, it is useful to learn a bit about the R syntax here. As with all programming problems, the key is to break up this cumbersome command into its parts.

The first line uses the logical function `==`, but on the left hand side, we are asking for just the month column of flights.

`flights[]` is just the whole flights dataset, and inside the square brackets are a comma. What do you think we get when we type `flights[ 4 , 4 ]` ?

Were you right? Try changing the numbers.

As you can see, we have a way to get elements out of the dataframe. What do you think we get when we write,

- `flights[ , "dest" ]`

or

- `flights[ , "day" ]`

Remember, rows and columns.

Now let's say we want the 4th entry under month, we could just write

- `flights[4 , "month" ]`

All of this indexing assumes we know in advance what index we are looking for. In practical terms though, we want to be able to search through the data to find specific entries.

For the next couple exercises, let's work with a subset of the data, let's examine flight patterns from New York into DC on January 18-21. We start with the way that we were able to make it before, then we redo it with some new commands. `c( )` makes a vector, and `A %in% B` is yet another logical command that reports TRUE for each element of A present somewhere in B. The `setdiff` command shows that there are no elements of the second dataset not present in the first. Formally, `setdiff` returns the set $\{x : x \in A \ \& \ x \notin B\}$.

```
DCflights <- filter(flights,
  month == 1,
  day == 18 | day == 19 | day == 20 | day == 21,
  dest == "IAD" | dest == "DCA")

DCflightsMethod2 <- filter(flights,
  month == 1,
  day %in% 18:21,
  dest %in% c("IAD", "DCA") )

setdiff(DCflights, DCflightsMethod2)

## # A tibble: 0 x 19
## # ... with 19 variables: year <int>, month <int>, day <int>,
## #   dep_time <int>, sched_dep_time <int>, dep_delay <dbl>, arr_time <int>,
## #   sched_arr_time <int>, arr_delay <dbl>, carrier <chr>, flight <int>,
```

```
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
```

Within the DC flights, suppose we want to get the longest flights. First we need to grab the air time variable, which we could do using the `DCflights[, "air_time"]`

As a short cut, R allows us to use the `$` operator to get named variables from a dataset. Rather than `DCflights[, "air_time"]` we can write the faster `DCflights$air_time`

Now we want the max of this vector. Note we need a separate command for how to deal with missing data indicated by NA. `na.rm` means remove NA values, and then do the calculation.

```
max(DCflights$air_time, na.rm=T)
```

```
## [1] 61
```

It seems that the max is 61. Lets find out where that is true.

```
DCflights$air_time==61
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [12] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [23] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [34] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [45] FALSE FALSE FALSE FALSE FALSE NA FALSE FALSE FALSE FALSE FALSE
## [56] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [67] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [78] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [89] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [100] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [111] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [122] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [144] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [155] FALSE FALSE FALSE FALSE FALSE FALSE FALSE NA
```

What we really want is the 'index' of the flights that are 61 minutes long. For that we can use the `which()` command

```
which(DCflights$air_time == 61)
```

```
## [1] 92
```


Five ways to get row 92

This gave us the index of the dataset where this is true, here the 92nd row. Finally, we can use this information in five ways to get that row of the dataset:

```
DCflights[92, ]
```

```
## # A tibble: 1 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
## 1  2013     1    20    1421           1429        -8    1552
## # ... with 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dtm>
```

```
DCflights[which(DCflights$air_time == 61), ]
```

```
## # A tibble: 1 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
## 1  2013     1    20    1421           1429        -8    1552
## # ... with 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dtm>
```

```
DCflights[DCflights$air_time == 61, ]
```

```
## # A tibble: 3 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
## 1    NA    NA    NA      NA              NA         NA      NA
## 2  2013     1    20    1421           1429        -8    1552
## 3    NA    NA    NA      NA              NA         NA      NA
## # ... with 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dtm>
```

```
DCflights[DCflights$air_time == max(DCflights$air_time, na.rm=T), ]
```

```
## # A tibble: 3 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
## 1    NA    NA    NA      NA              NA         NA      NA
## 2  2013     1    20    1421           1429        -8    1552
## 3    NA    NA    NA      NA              NA         NA      NA
## # ... with 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dtm>
```

```
filter(DCflights, air_time==max(air_time, na.rm=T))
```

```
## # A tibble: 1 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
```

```
## 1 2013      1    20      1421          1429      -8      1552
## # ... with 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dtm>
```

Note the second and third techniques also return the NA rows. This is because the `==` returns a NA, and putting in a NA in an index in R returns an NA.

An aside on indexing

One note on indexing, it is very powerful. Take the following vector, for example. We begin by creating an object, called `lengths`, which contains the words `short`, `medium`, and `long`. We then show that repeated indexes return repeated elements of the index.

```
lengths <- c("short", "medium", "long")
```

```
lengths[c(1,1,1,1,1,1,2,2,2,3,3,3,1,1,2,3)]
```

```
## [1] "short" "short" "short" "short" "short" "short" "medium"
## [8] "medium" "medium" "long"  "long"  "long"  "short" "short"
## [15] "medium" "long"
```

```
# Suppose we make some fake data:
```

```
(fake_responses <- rbinom(10, size = 2, prob = 0.3)+1)
```

```
## [1] 2 1 1 2 2 2 2 1 2 2
```

```
lengths[fake_responses]
```

```
## [1] "medium" "short"  "short"  "medium" "medium" "medium" "medium"
## [8] "short"  "medium" "medium"
```

Christmas Eve Analysis

Now, I am not sure if you have ever taken a flight on Christmas Eve, but the pilots often fly by one another with their lights set up in a Rudolf leading the reindeer, with a red light up front, and they make an announcement for the children.

```
filter(flights, month==12, day ==24 )
```

```
## # A tibble: 761 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
## 1  2013    12    24         9           2359         10     444
## 2  2013    12    24       458           500         -2     652
## 3  2013    12    24       513           515         -2     813
## 4  2013    12    24       543           540          3     844
## 5  2013    12    24       546           550         -4    1032
## 6  2013    12    24       555           600         -5     851
## 7  2013    12    24       556           600         -4     845
## 8  2013    12    24       557           600         -3     908
## 9  2013    12    24       558           600         -2     827
##10  2013    12    24       558           600         -2     729
## # ... with 751 more rows, and 12 more variables: sched_arr_time <int>,
## #   arr_delay <dbl>, carrier <chr>, flight <int>, tailnum <chr>,
## #   origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
## #   minute <dbl>, time_hour <dtm>
```

Filter to the flights at night, let's say after 7:00 PM, or 1900

```
ChristmasEveFlights <- filter(flights, month==12, day ==24, dep_time > 1900 )
```

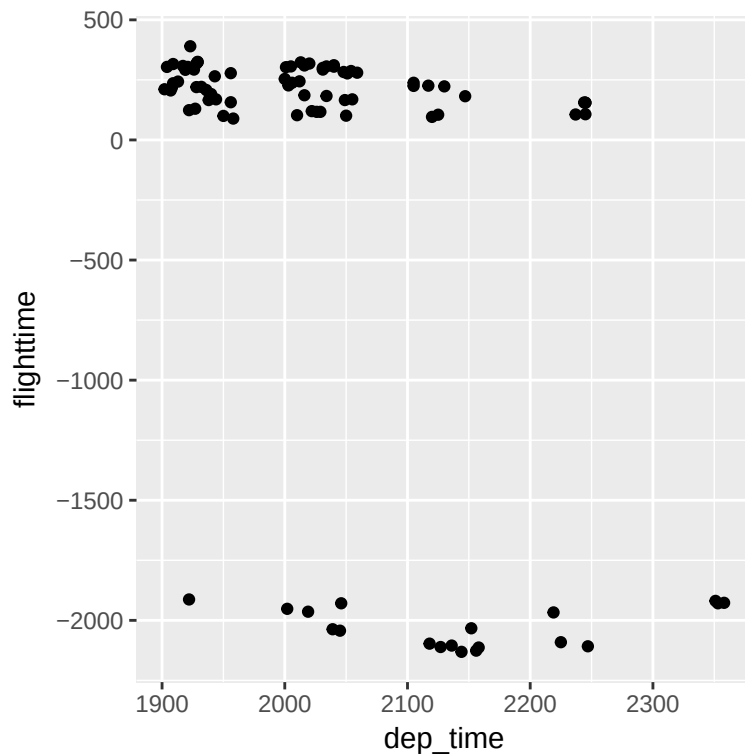
Now let's see who was in the air long enough to see the reindeer. To do so, we will add a new variable to the dataset, flighttime, by subtracting deptime from arrtime. We will add that variable using the `mutate()` function, which works just like filter, but instead of returning rows, it adds columns. You name the new variable whatever you like, and you use an `=` then the content of the new variable.

```
ChristmasEveFlights <- mutate(ChristmasEveFlights, flighttime = arr_time - dep_time)
```

Plotting Flights

Lets use a simple plot to visualize these flights. Remember you can learn about all commands using `?qplot`

```
qplot(x = dep_time, y = flighttime, data = ChristmasEveFlights)
```

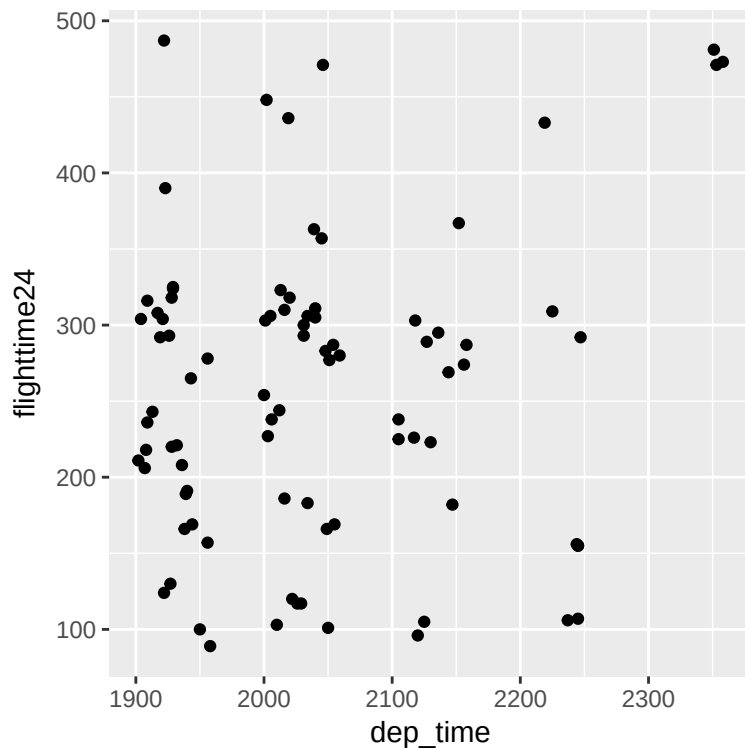


It seems like something is wrong. Some flights are spilling over into the next day. Let's return to the Christmas Eve Flights equation. What do we need to do? We want to add 2400 hours when the flighttime is negative. We can do that with an `ifelse` command. `ifelse` commands take a logical statment and return the second element if it is true and the third element if it is false.

```
ifelse(2==3, "Is correct", "Is not correct")
```

```
## [1] "Is not correct"
```

```
ChristmasEveFlights <- mutate(ChristmasEveFlights,  
  flighttime = arr_time - dep_time,  
  flighttime24 = ifelse( flighttime < 0, flighttime+2400, flighttime))  
qplot(dep_time, flighttime24, data=ChristmasEveFlights)
```



Summarizing the Data

Is this an atypical distribution? Let's get some simple statistics from these data.

```
summary(ChristmasEveFlights)
```

```
##      year      month      day      dep_time      sched_dep_time
## Min.   :2013   Min.   :12   Min.   :24   Min.   :1902   Min.   :1530
## 1st Qu.:2013   1st Qu.:12   1st Qu.:24   1st Qu.:1938   1st Qu.:1930
## Median :2013   Median :12   Median :24   Median :2022   Median :2021
## Mean   :2013   Mean   :12   Mean   :24   Mean   :2041   Mean   :2020
## 3rd Qu.:2013   3rd Qu.:12   3rd Qu.:24   3rd Qu.:2118   3rd Qu.:2116
## Max.   :2013   Max.   :12   Max.   :24   Max.   :2358   Max.   :2359
##      dep_delay      arr_time      sched_arr_time      arr_delay
## Min.   : -14.00   Min.   : 2   Min.   : 3   Min.   : -46.000
## 1st Qu.: -5.00   1st Qu.:2054   1st Qu.:1962   1st Qu.: -16.000
## Median : -1.00   Median :2208   Median :2203   Median : -6.000
## Mean   : 12.64   Mean   :1773   Mean   :1738   Mean   : 3.609
## 3rd Qu.: 8.50   3rd Qu.:2318   3rd Qu.:2281   3rd Qu.: 7.500
## Max.   :286.00   Max.   :2400   Max.   :2359   Max.   :251.000
##      carrier      flight      tailnum      origin
## Length:87      Min.   : 21.0   Length:87      Length:87
## Class :character 1st Qu.: 437.5   Class :character Class :character
## Mode :character  Median :1203.0   Mode :character Mode :character
##                      Mean   :1397.1
##                      3rd Qu.:2149.0
##                      Max.   :4113.0
##      dest      air_time      distance      hour
## Length:87      Min.   : 33.0   Min.   : 187.0   Min.   :15.0
```

```
## Class :character 1st Qu.: 86.0 1st Qu.: 515.5 1st Qu.:19.0
## Mode :character Median :146.0 Median :1023.0 Median :20.0
## Mean :162.6 Mean :1129.7 Mean :19.9
## 3rd Qu.:199.0 3rd Qu.:1598.0 3rd Qu.:21.0
## Max. :352.0 Max. :2586.0 Max. :23.0
## minute time_hour flighttime
## Min. : 0.00 Min. :2013-12-24 15:00:00 Min. : -2131.0
## 1st Qu.:16.00 1st Qu.:2013-12-24 19:00:00 1st Qu.: 100.5
## Median :30.00 Median :2013-12-24 20:00:00 Median : 189.0
## Mean :30.23 Mean :2013-12-24 19:53:47 Mean : -267.8
## 3rd Qu.:45.00 3rd Qu.:2013-12-24 21:00:00 3rd Qu.: 281.5
## Max. :59.00 Max. :2013-12-24 23:00:00 Max. : 390.0
## flighttime24
## Min. : 89.0
## 1st Qu.:182.5
## Median :269.0
## Mean :256.3
## 3rd Qu.:307.0
## Max. :487.0
```

The average flight time on Christmas Eve is 256. For comparison, we should do the same calculation for the whole dataset.

```
flights <- mutate(flights,
  flighttime = arr_time - dep_time,
  flighttime24 = ifelse( flighttime < 0, flighttime+2400, flighttime))
summary(flights$flighttime24)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
##      35     166     224     231    294    1925    8713
```

The average for the whole population is 231. Is this the right way to summarise this data? We can add a variable to the flights data that indicates that the flight is a Christmas eve flight, so we don't have to keep messing with two datasets.

```
flightscomparison <- mutate(flights,
  ChristmasEve = ifelse( month==12 & day==24& dep_time>1900,
    "Christmas Eve", "Un-Christmas Eve"))
```

Let further limit the analysis to evenings evenings, we don't want our analysis messed up by morning flights.

```
flightscomparison <- filter(flightscomparison, dep_time>1900)
head(flightscomparison)
```

```
## # A tibble: 6 x 22
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>
## 1  2013     1     1    1904           1905          -1    2139
## 2  2013     1     1    1904           1905          -1    2157
## 3  2013     1     1    1905           1900           5    2311
## 4  2013     1     1    1906           1915          -9    2211
## 5  2013     1     1    1909           1912          -3    2239
## 6  2013     1     1    1909           1910          -1    2212
```

```
## # ... with 15 more variables: sched_arr_time <int>, arr_delay <dbl>,  
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,  
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,  
## #   time_hour <dtm>, flighttime <int>, flighttime24 <dbl>,  
## #   ChristmasEve <chr>
```

Advanced Plotting

After you have mastered the above commands, we can turn to advanced plotting techniques, from `ggplot2`, the most cutting edge data visualization software on the market. Learning about `ggplot2` will be a substantial part of this course, and a great tool you will use for the rest of your careers.

`ggplot2` has a particular philosophy about how to plot. The basic parts are a `ggplot()` command to input the data, and then layers or `geoms` are added with `+` signs afterwards.

Inside a `ggplot` function we start with some data, and then we tell the computer which rows and columns from the data are to be used as the x and y axis. This is done inside a function called an aesthetic command `aes()`. The neat thing is that just like `mutate` and `filter`, the aesthetic command does not need quotation marks, more on that later.

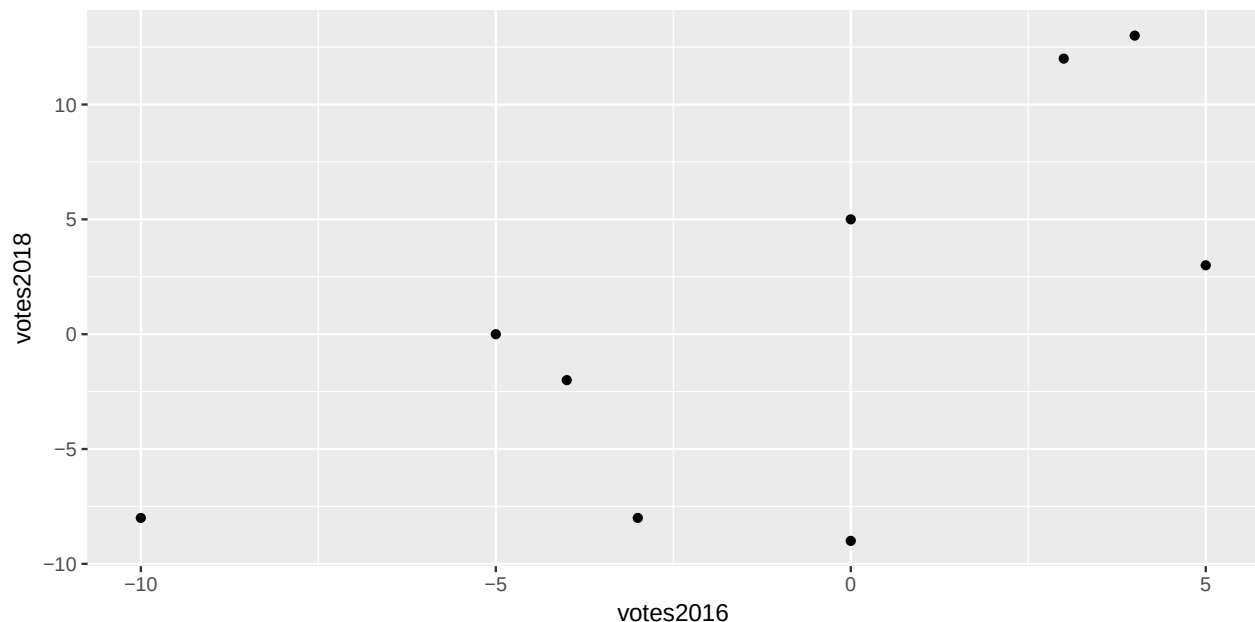
The next thing to do is add a layer, or `geom`, which tells `ggplot` what shape you want the data to take. Here is a list of the ones I use most often: `+ geom_points` : Common scatterplot. Takes on a `x` and `y` `aes`. `+ geom_bar` : Barplot, great for categorical data. `+ geom_histogram` : A histogram, which shows the proportion or frequency of a continuous variable in various bins. `+ geom_density` : Displays a smoothed histogram (essentially). `+ geom_smooth` : Adds a regression line to a scatterplot. Defaults to a smoothed local regression. `+ geom_text` : like a scatterplot but with text.

These commands will show up throughout the course.

We start with a very simple dataframe and plot:

```
votes2016<-c(-5, -4, -3, -10, 0, 3, 4, 0, 5)
votes2018<-c(0, -2, -8, -8, -9, 12, 13, 5, 3)
df<-data.frame(votes2016=votes2016, votes2018=votes2018)

ggplot(df, aes(x = votes2016, y = votes2018)) +
  geom_point()
```

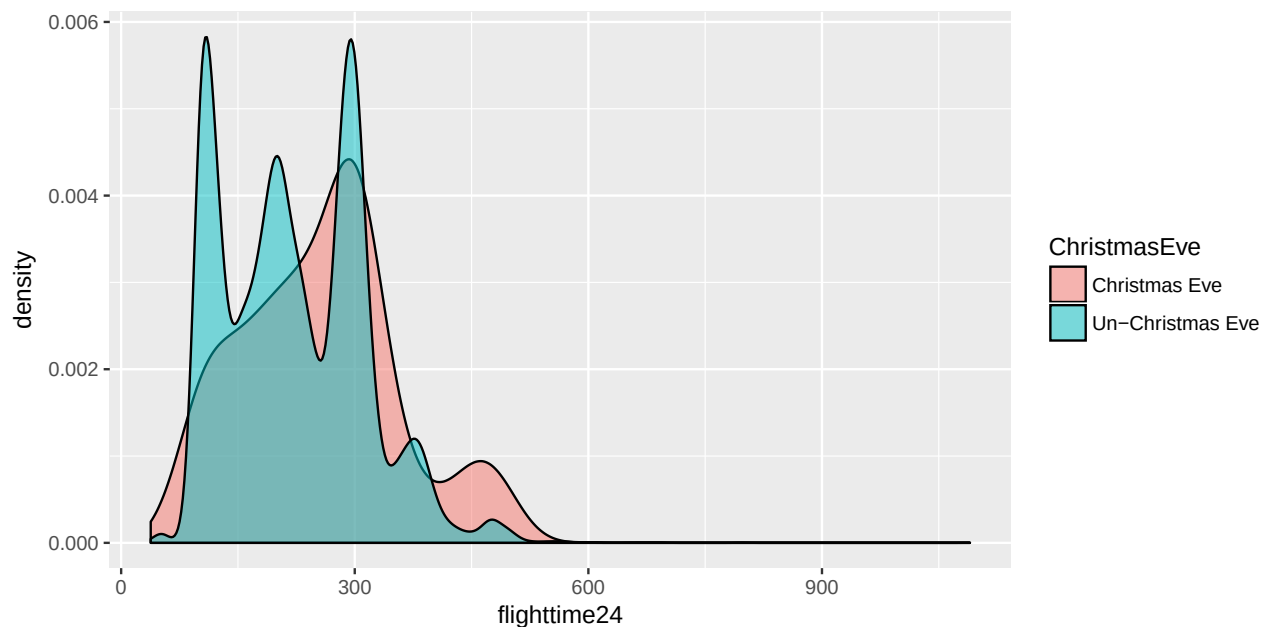


Simple, clear, attractive. Now lets consider a more advanced plot.

The `aes` is not just for 2 dimension plots, it can be used to specify all sorts of interesting 3rd and 4th dimensions of the data using the `group` and `fill` commands.

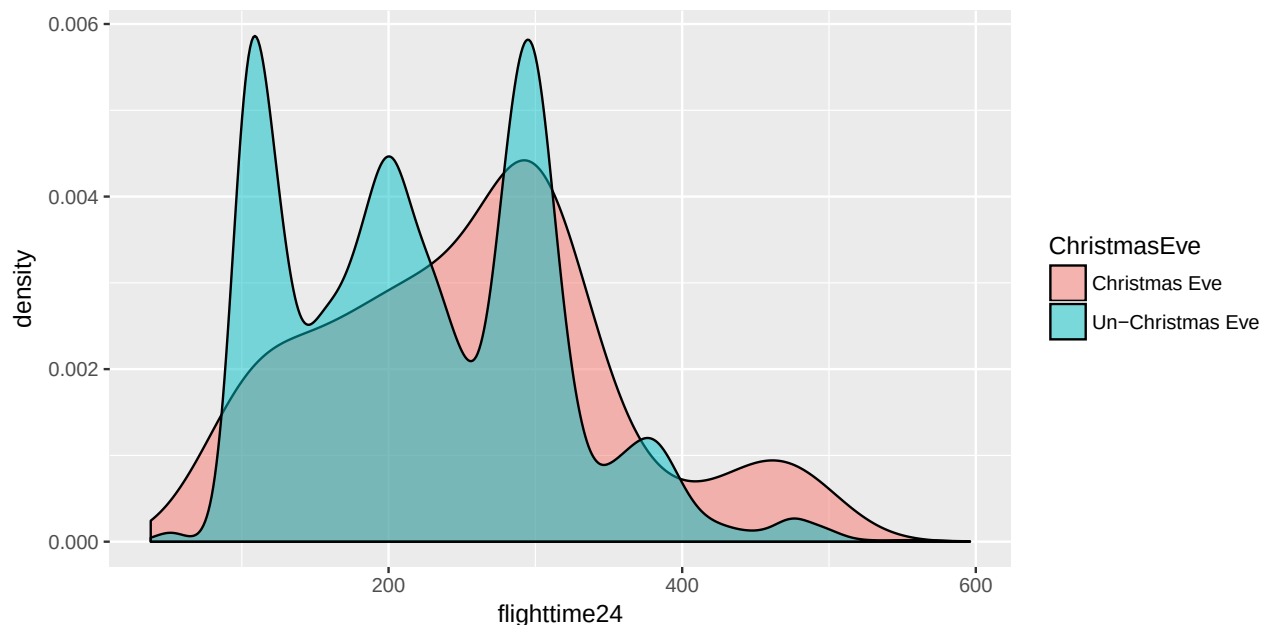
```
ggplot(flightscomparison,  
  aes(flighttime24, group = ChristmasEve, fill= ChristmasEve ))+  
  geom_density(alpha=.5)
```

Warning: Removed 117 rows containing non-finite values (stat_density).



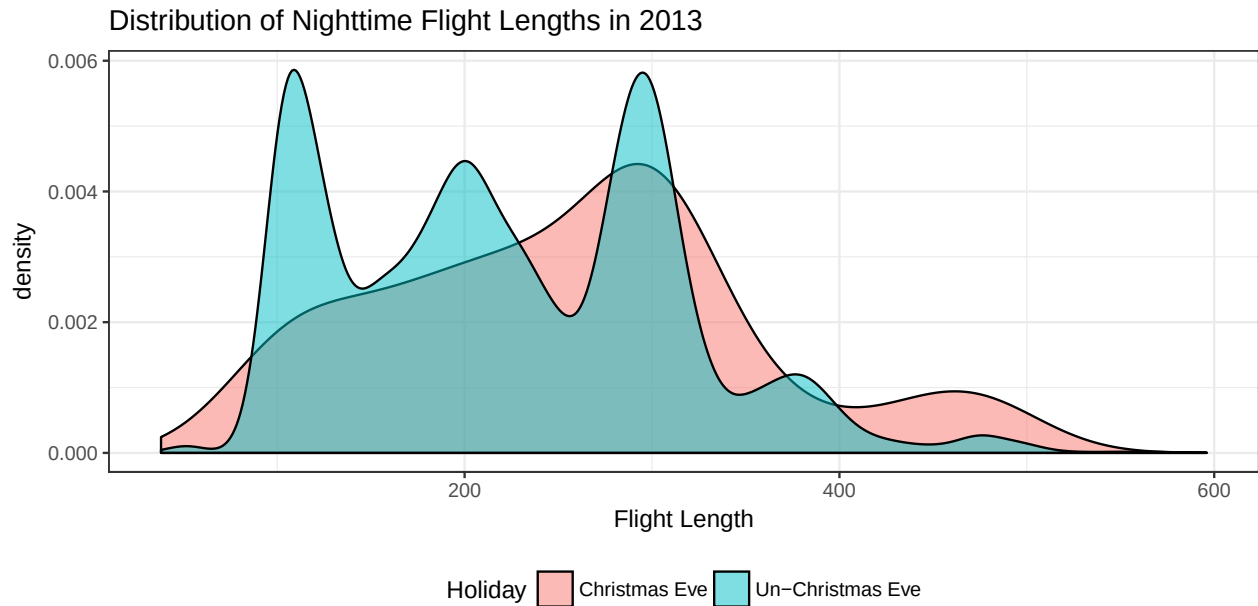
This is a bit hard to read. To improve legibility, we can cut out all flights more than 6 hours. Note that we are putting the filter command right into the ggplot command.

```
ggplot(filter(flightscomparison, flighttime24<600),  
  aes(flighttime24, group = ChristmasEve, fill= ChristmasEve ))+  
  geom_density(alpha=.5)
```



For your enjoyment, I am going to add some features to the plot above, to make it look nice.

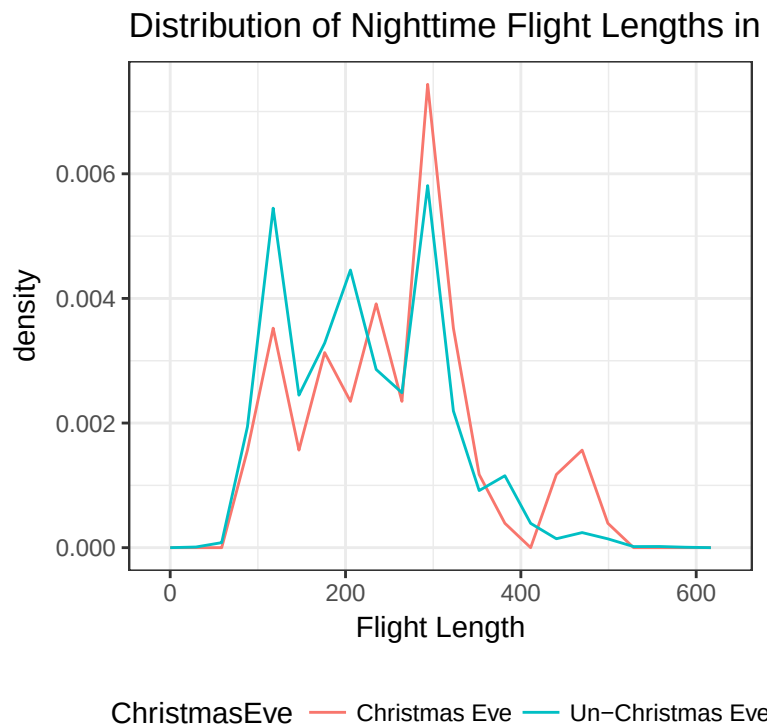
```
ggplot( filter(flightscomparison, flighttime24<600),
  aes(flighttime24, group = ChristmasEve, fill= ChristmasEve ))+
geom_density(alpha=.5)+
theme_bw()+
xlab("Flight Length")+
ggtitle("Distribution of Nighttime Flight Lengths in 2013")+
guides(fill = guide_legend(title = "Holiday"))+theme(legend.position="bottom")
```



Histogram plot

For comparison, see the following histogram plot. This histogram shows the proportion of the data that occurs in bins of a continuous variable. I am using the new `freqpoly` geom, which draws a line at the top of the various histogram bins.

```
ggplot( filter(flightscomparison, flighttime24<600),
  aes(flighttime24, group = ChristmasEve, color = ChristmasEve ))+
geom_freqpoly(bins=20, aes(y=..density..), position="identity")+
theme_bw()+
xlab("Flight Length")+
ggtitle("Distribution of Nighttime Flight Lengths in 2013")+
guides(fill = guide_legend(title = "Holiday"))+theme(legend.position="bottom")
```

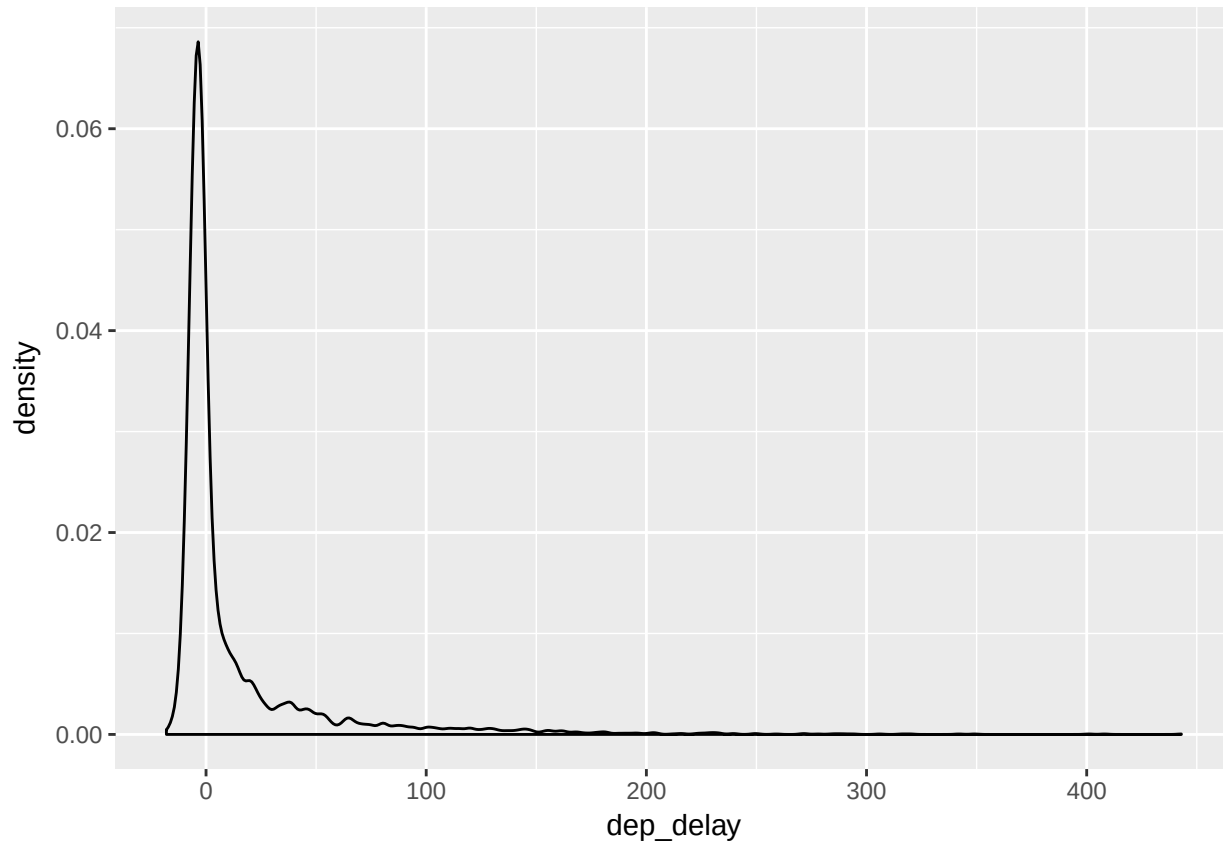


Plotting skewed data

Very often we transform our data to improve visualization, not just to fit our concepts better. One of those transforms used very often is the logarithm. Consider the distribution of departure delay for a sample of 5000 flights:

```
ggplot(sample_n(flights, 5000), aes(dep_delay))+geom_density()
```

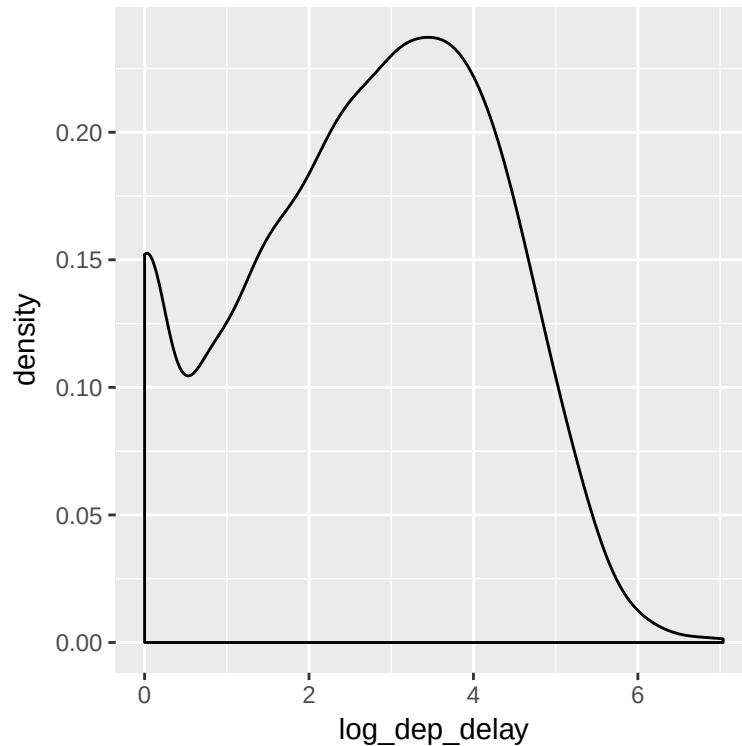
```
## Warning: Removed 101 rows containing non-finite values (stat_density).
```



A few points really stretch out the x axis. If we take the log, we can get a better look. To do so, we first remove observations less than or equal to 0.

```
samp_flights      <- sample_n(flights, 5000) # This draws a random 5000 rows from flights
samp_flights_delay <- filter(samp_flights, dep_delay>=0) # This removes negative delay
samp_flights_delay <- mutate(samp_flights_delay, log_dep_delay = log(dep_delay+1))

ggplot(samp_flights_delay, aes(log_dep_delay))+geom_density()
```



Writing data in R is simple. Here we save this data that we made to do statistical inference later in the quarter. Again, put the path in a folder on your computer.

```
write.csv(flightscomparison, "~/Dropbox/mathCamp/Code/data/flightcomparison.csv")
```